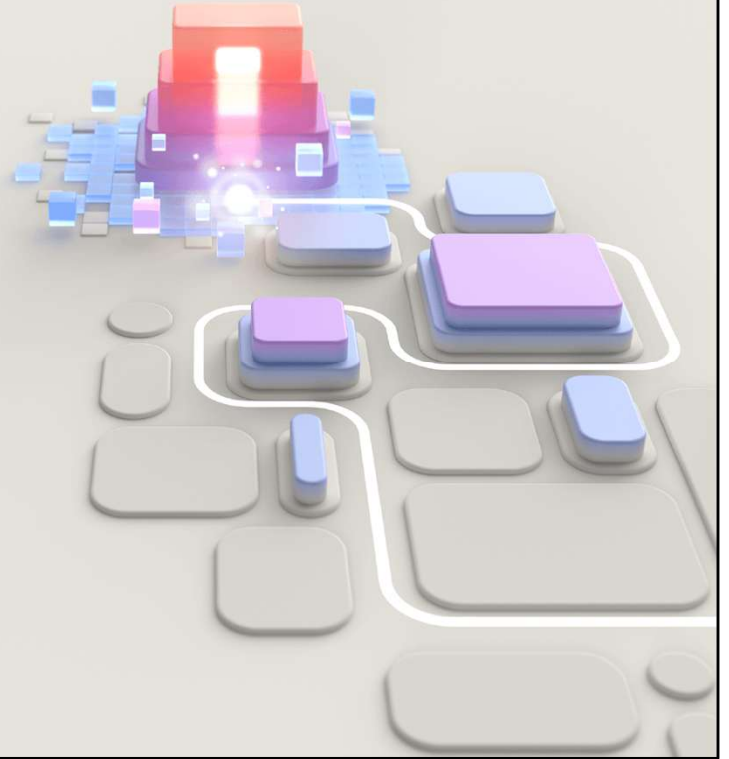




DP-300

Implement scalable database solutions using Azure SQL



© Copyright Microsoft Corporation. All rights reserved.

This should be present in the first slide deck of every workshop.

Optimize query performance in Azure SQL

<https://learn.microsoft.com/training/paths/optimize-query-performance-sql-server/>

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/paths/optimize-query-performance-sql-server/>

Objectives

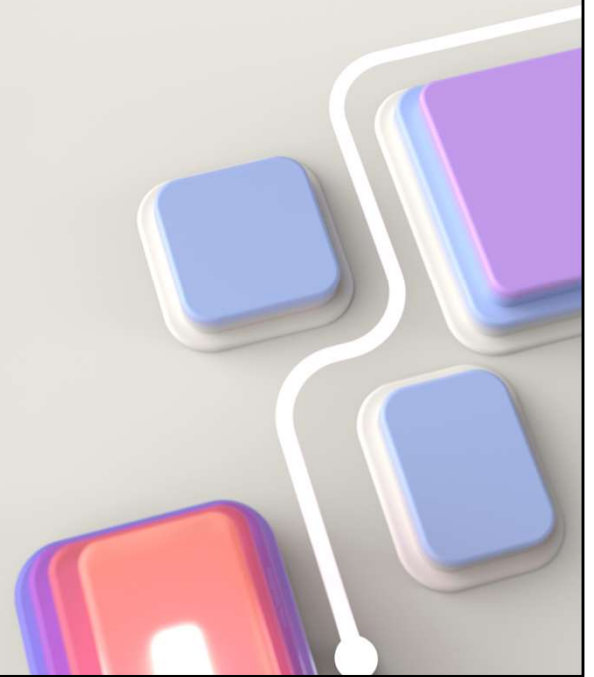
- Analyze query plans and identify problem areas
- Evaluate potential query improvements
- Review table and index design
- Determine whether query or design changes have had a positive effect

© Copyright Microsoft Corporation. All rights reserved.

Explore query performance optimization

<https://learn.microsoft.com/training/modules/explore-query-performance-optimization/>

© Copyright Microsoft Corporation. All rights reserved.



Objectives

-  Generate and save execution plans
-  Compare the different types of execution plans
-  Understand how and why query plans are generated
-  Understand the purpose and benefits of the Query Store

© Copyright Microsoft Corporation. All rights reserved.

The most important skill you should acquire in database performance tuning is being able to read and understand query execution plans. The plans explain the behavior of the database engine as it executes queries and retrieves the results.

Query Store helps you quickly identify your most expensive queries, and find any changes in performance. By automating the collection of query plan and execution runtime, the Query Store provides a powerful complete data collection. SQL Server and Azure SQL provide locking and blocking in order to manage concurrency and ensure data consistency. Finally, you can adjust the isolation levels in SQL Server to help manage concurrency.

What are execution plans?



Each time you run a query, the query optimizer determines the most efficient sequence of operations to process your request and fetch the results.

The optimizer also collects statistics about the operation and execution of this plan.

How are query plans generated?

The first time you run a given query, the optimizer generates several plans and chooses the one with the lowest overall cost.

© Copyright Microsoft Corporation. All rights reserved.

What are execution plans?

- The optimizer also collects statistics about the operation and execution of this plan
- After being generated by the optimizer, query plans are cached in an area of memory called the **plan cache**
- Stored internally as XML

Query execution plans are arguably the most important aspect of database performance. Improving bad plans is certainly an area where a small amount of effort can bring huge improvements. While hardware issues can limit query performance, improving hardware usually yields performance improvements in the 10-20% range. More commonly database administrators encounter queries that are not optimized, have stale or missing statistics, have missing indexes, or poor database design choices that lead to the database engine doing more work than is necessary to return results for a given query. Improving the plans can sometimes yield performance improvements in the 100-200% range or even more, meaning that after improving a plan with better indexes or statistics, a query could run twice or three times as fast!

How are query plans generated?

- The total cost of the execution plan is a blended measure of how much resource consumption the optimizer thinks will be needed to complete the query
- The optimizer uses settings and distribution statistics on columns and indexes associated with your query to generate the execution plans
- Generating plans is very resource intensive, so plans are cached after initial generation

The query is then executed using an execution plan is pulled either from the cache, or a new plan generated in step 4. The output of this step is the results of your query.

For the example query:

```
SELECT orderdate
      ,avg(salesAmount)
FROM FactResellerSales
WHERE ShipDate = '2013-07-07'
GROUP BY orderdate;
```

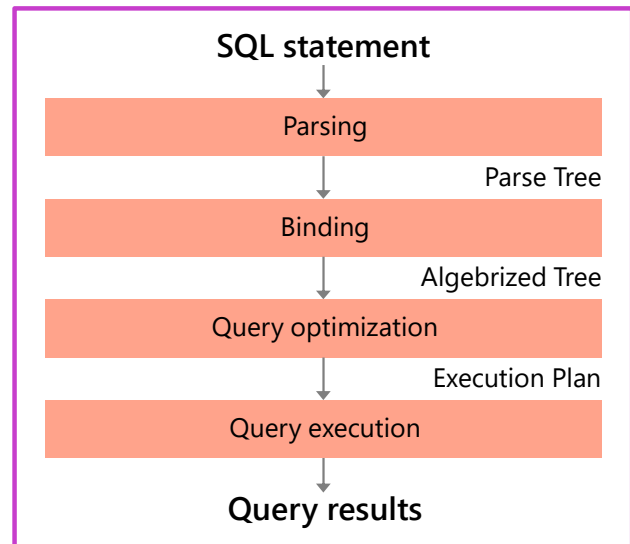
In this example SQL Server will check for the existence of the OrderDate, ShipDate and SalesAmount columns in the table FactResellerSales. If those columns exist it will then generate a hash value for the query and examine the plan cache for a matching hash value. If there is plan for a query with a matching hash the engine will reuse that plan, if not it will examine the statistics it has available on the OrderDate column. The WHERE clause referencing the ShipDate column is what is known as the predicate in this query. If there is a nonclustered index that includes the ShipDate column SQL Server will most likely include that in the plan, if the costs are lower than retrieving data from the clustered index. The optimizer will then choose the lowest cost plan of the available plans and execute the query.

Query plans combine a series of relational data operators to retrieve the data, and also capture information about the data such as estimated row counts. Another element of the execution plan is the memory required to perform operations such as joining or sorting data, which is known as the memory grant. The memory grant is a good example of the importance of statistics. If SQL Server thinks an operator is going to return 10,000,000 rows, when it is only returning 100, a much larger amount of memory is granted to the query. This can cause a twofold problem—the query may encounter a RESOURCE_SEMAPHORE wait, which indicates that query is waiting for SQL Server to allocate it a large amount of memory. SQL Server defaults to waiting for 25 times the cost of the query (in seconds) before executing, up to 24 hours. When the query is executed, if there is not enough memory available the query will spill to tempDB, which is much slower than operating in memory.

The execution plan also stores other metadata about the query, including, but not limited to, the database compatibility level, the degree of parallelism of the query, and the parameters that are supplied if the query is parameterized.

The query optimizer

- The query optimizer takes a query and returns a query plan
- Will attempt several plans to choose the *'best guess'*
- Generating a plan is CPU-intensive; SQL Server caches them.



© Copyright Microsoft Corporation. All rights reserved.

A query that gets executed, goes through a process to generate an execution plan and return results.

Parsing and binding – the query is parsed and bound. Assuming the query is valid, the output of this phase is a logical tree, with each node in the tree representing a logical operation that the query must perform, such as reading a particular table, or performing an inner join. This logical tree is then used to run the query optimization process, which roughly consists of the following two steps;

Generate possible execution plans – using the logical tree, the Query Optimizer devises a number of possible ways to execute the query i.e. a number of possible execution plans. An execution plan is, in essence, a set of physical operations (an index seek, a nested loop join, and so on), that can be performed to produce the required result, as described by the logical tree;

Cost-assessment of each plan – While the Query Optimizer does not generate every possible execution plan, it assesses the resource and time cost of each plan it does generate. The plan that the Query Optimizer deems to have the lowest cost of those it's assessed is selected, and passed along to the Execution Engine;

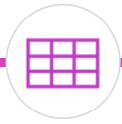
Query execution, plan caching – the query is executed by the Execution Engine, according to the selected plan. The plan may be stored in memory, in the plan cache.

Types of execution plans



Estimated execution plans

The path the database engine plans to take to retrieve the data to answer your query



Actual execution plans

The estimated execution plan, but also including actual statistics from the execution of the query



Live query statistics

An animated view of query execution showing progress and order of operations

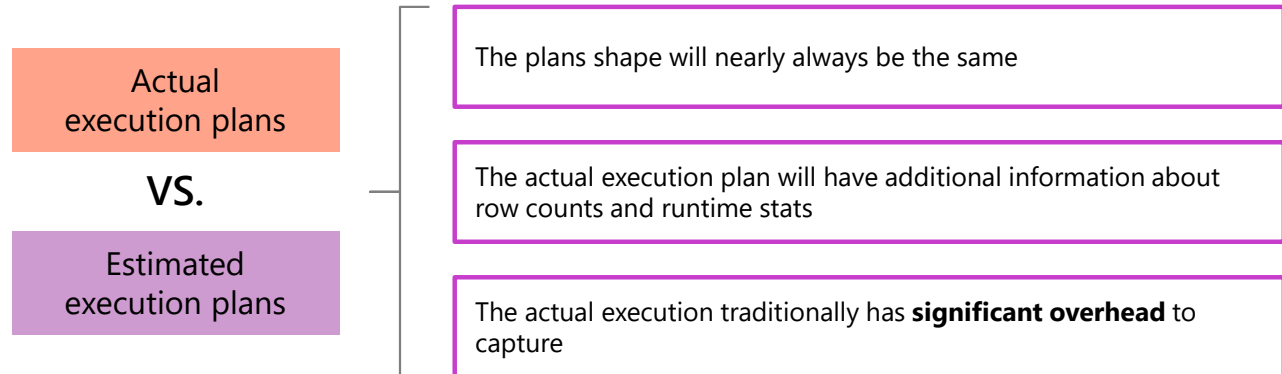
© Copyright Microsoft Corporation. All rights reserved.

Estimated Execution Plan: This is the execution plan as generated by the query optimizer. The metadata and size of query memory grant are based on estimates from the statistics as they exist in the database at the time of query compilation.

Actual Execution Plan: This is same plan as the estimated plan; however this plan also contains the execution context for the query which includes the estimated and actual row counts, any execution warnings, and elapsed and CPU times used during the execution.

Live Query Statistics : This plan viewing option combines the estimated and actual plans into an animated plan that displays execution progress through the operators in the plan. It refreshes every second and shows the actual number of rows flowing through the operators. The other benefit to Live Query Statistics is that it shows the handoff from operator to operator which may be helpful in troubleshooting some performance issues.

More about actual vs. estimated execution plans



© Copyright Microsoft Corporation. All rights reserved.

Bilden förklarar skillnaden mellan **Actual execution plan** och **Estimated execution plan** i SQL Server. Båda planerna får i princip **samma form/struktur**, eftersom optimeraren väljer samma strategi (t.ex. join-typ och index). Skillnaden är att **Actual plan** innehåller extra information från körningen, t.ex. **faktiskt antal rader (Actual Rows)**, **körtider**, varningar och runtime-statistik. Det gör den mycket bättre för felsökning när prestanda är dålig eller när skattningar är fel. Nackdelen är att Actual plan kräver att frågan verkligen körs och kan ge **påtaglig overhead**, särskilt på tunga frågor. Estimated plan är snabbare och säkrare att ta fram.

The topic of actual versus estimated execution plans can be confusing. As mentioned above, the difference is that the actual plan includes runtime statistics that are not captured in the estimated plan. The operators used, and order of execution will be the same as the estimated plan in nearly all cases. The other consideration is that in order to capture an actual execution plan the query has to be actually executed, and this may be extremely time consuming, or not be possible (e.g. an UPDATE statement that can only be run once). However, if you need to see query results as well as the plan, you'll need to use one of the actual plan options.

Lightweight query profiling



Supported by Azure SQL Database



Required trace flag 7412 to be enabled globally, prior to SQL Server 2019



Newer releases allow it to be enabled at the query level



Provides additional statistics to help close gap to actual execution plan data

© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver **Lightweight Query Profiling**, en funktion som ger extra körningsstatistik för frågor med **lägre overhead** än att alltid samla in full "Actual execution plan"-data. Funktionen stöds i **Azure SQL Database**, vilket gör den praktisk när man vill felsöka prestanda i en PaaS-miljö utan att belasta systemet för mycket. I äldre SQL Server-versioner (före **SQL Server 2019**) krävdes att man slog på **trace flag 7412 globalt** för att aktivera den här typen av profilering. I nyare versioner kan den istället ofta aktiveras mer selektivt, t.ex. på **query-nivå**, vilket är säkrare i produktion. Syftet är att minska gapet mellan Estimated plan och Actual plan genom att ge mer runtime-information, t.ex. radflöden, operator-statistik och annan körningsdata som hjälper dig hitta flaskhalsar och felaktiga kardinalitetsskattningar.

```
SELECT *
FROM Sales.SalesOrderDetail
WHERE ProductID = 870
OPTION (USE HINT('QUERY_PLAN_PROFILE'));
```

```
SELECT *
FROM sys.dm_exec_query_profiles;
```

As mentioned above, capturing actual execution plans, whether using SSMS or the Extended Events monitoring infrastructure (to be covered in more detail later) can have a large amount of overhead, and is typically only done in live site troubleshooting efforts. Observer overhead, as it is known, is the cost of monitoring a running application. In some scenarios this can be just a few percentage points of CPU utilization, but in other cases like capturing actual execution plans, it can slow down individual query performance significantly. The legacy profiling infrastructure in SQL Server's engine could produce up to 75% overhead for capturing query information, whereas the lightweight profiling infrastructure has a maximum overhead of around 2%.

How to read an execution plan



Plans should be read from right to left and top to bottom



The **size** of the arrows connecting the objects is representative of the amount of data flowing to the next operator



Each operator contains a lot of metadata in its properties and tool tip. This is where you can learn how the decisions were made



All operators are assigned a cost

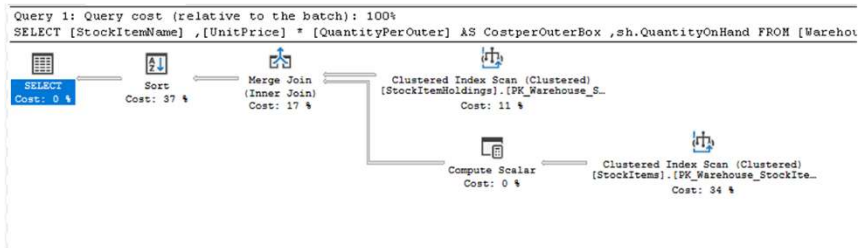


Even operators that have a 0% cost have some degree of cost – Nothing is free in SQL Server query execution

© Copyright Microsoft Corporation. All rights reserved.

Example: SQL query with a JOIN clause

```
SELECT [stockItemName],
       [UnitPrice] * [QuantityPerOuter] AS CostPerOuterBox,
       [QuantityOnHand]
FROM [Warehouse].[StockItems] s
     JOIN [Warehouse].[StockItemHoldings] sh ON s.StockItemID = sh.StockItemID
ORDER BY CostPerOuterBox;
```



© Copyright Microsoft Corporation. All rights reserved.

Tool tips and properties

Clustered Index Scan (Clustered)
[StockItems].[PK_Warehouse_StockItem...]
Cost: 34 %

Clustered Index Scan (Clustered)	
Scanning a clustered index, entirely or only a range.	
Physical Operation	Clustered Index Scan
Logical Operation	Clustered Index Scan
Estimated Execution Mode	Row
Storage	Row/Store
Estimated Operator Cost	0.0131613 (34%)
Estimated I/O Cost	0.0127546
Estimated Subtree Cost	0.0131613
Estimated CPU Cost	0.0004067
Estimated Number of Executions	1
Estimated Number of Rows	227
Estimated Number of Rows to be Read	227
Estimated Row Size	128 B
Ordered	True
Node ID	4
Object	[WideWorldImporters].[Warehouse].[StockItems].
	[PK_Warehouse_StockItems] [s]
Output List	[WideWorldImporters].[Warehouse].[StockItems].StockItemID,
	[WideWorldImporters].[Warehouse].[StockItems].StockItemName,
	[WideWorldImporters].[Warehouse].
	[StockItems].QuantityPerOuter, [WideWorldImporters].
	[Warehouse].[StockItems].UnitPrice

Properties
Clustered Index Scan (Clustered)

Clustered Index Scan (Clustered)	
Misc	
Defined Values	[WideWorldImporters].[Warehouse].[StockItems].St...
Description	Scanning a clustered index, entirely or only a range.
Estimated CPU Cost	0.0004067
Estimated Execution Mode	Row
Estimated I/O Cost	0.0127546
Estimated Number of Executions	1
Estimated Number of Rows	227
Estimated Number of Rows to be Read	227
Estimated Operator Cost	0.0131613 (34%)
Estimated Rebinds	0
Estimated Row Size	128 B
Estimated Subtree Cost	0.0131613
Forced Index	False
ForceScan	False
ForceSeek	False
Logical Operation	Clustered Index Scan
Node ID	4
NoExpandHint	False
Object	[WideWorldImporters].[Warehouse].[StockItems].(PK_...
Alias	[s]
Database	[WideWorldImporters]
Index	[PK_Warehouse_StockItems]
Index Kind	Clustered
Schema	[Warehouse]
Storage	Row/Store
Table	[StockItems]
Ordered	True
Output List	[WideWorldImporters].[Warehouse].[StockItems].Stock
[1]	[WideWorldImporters].[Warehouse].[StockItems].Stock
[2]	[WideWorldImporters].[Warehouse].[StockItems].Stock
[3]	[WideWorldImporters].[Warehouse].[StockItems].Quar
[4]	[WideWorldImporters].[Warehouse].[StockItems].UnitP
Parallel	False
Physical Operation	Clustered Index Scan
Scan Direction	FORWARD
Storage	Row/Store
Table Cardinality	227

© Copyright Microsoft Corporation. All rights reserved.

The tooltip highlights the cost and estimates for estimated plan, and for an actual plan will include the comparisons to the actual rows and costs, and can be obtained by hovering over the execution plan operator.

The properties include some additional information and shows the output list which provides details of the columns being passed to the next operator. These columns, in conjunction with a clustered index scan operator can indicate that additional nonclustered index might be needed to improve the performance of the query. It can be obtained by right clicking on a query plan operator and selecting properties.

Common plan operators: Finding data

Index Seek

Reads the portion of the index which contains the needed data

Index Scan

Reads the entire index for the needed data

Table Scan

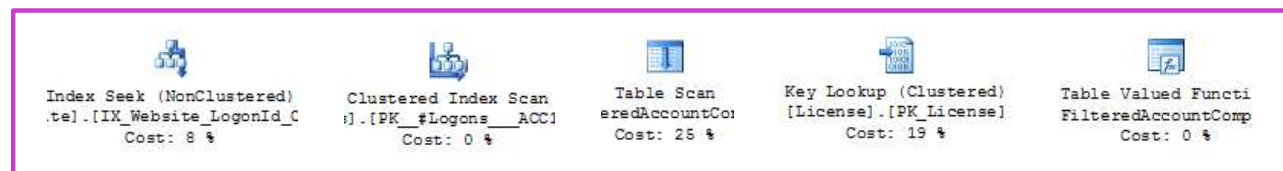
Reads the entire table for the needed data

Key Lookup

Looks up values row by row for values which are missing from the index used

Table Valued Function

Executes a table valued function within the database



© Copyright Microsoft Corporation. All rights reserved.

The **Index Seek** operator uses the seeking ability of indexes to retrieve rows from a nonclustered index. The **Argument** column contains the name of the nonclustered index being used. It also contains the SEEK:() predicate. The storage engine uses the index to process only those rows that satisfy the SEEK:() predicate.

The **Index Scan** operator retrieves all rows from the nonclustered index specified in the **Argument** column. If an optional WHERE:() predicate appears in the **Argument** column, only those rows that satisfy the predicate are returned. **Index Scan** is a logical and physical operator

The **Table Scan** operator retrieves all rows from the table specified in the **Argument** column of the query execution plan. If a WHERE:() predicate appears in the **Argument** column, only those rows that satisfy the predicate are returned.

The **Key Lookup** operator is a bookmark lookup on a table with a clustered index. The **Argument** column contains the name of the clustered index and the clustering key used to look up the row in the clustered index. **Key Lookup** is always accompanied by a **Nested Loops** operator.

The **Table-valued Function** operator evaluates a table-valued function (either Transact-SQL or CLR), and stores the resulting rows in the [tempdb](#) database. When the parent iterators request the rows, **Table-valued Function** returns the rows from **tempdb**.

Common plan operators: filtering & sorting

Nested Loops

Performs inner, outer, semi and anti semi joins. Performs search on the inner table for each row of the outer table

Hash Match

Creates a hash for required columns for each row. Then creates a hash for second table and finds matches

TOP

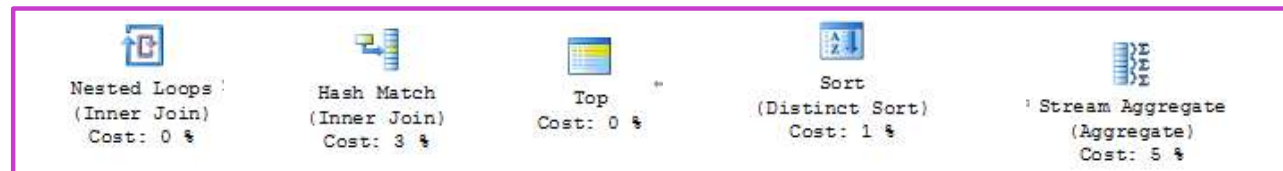
Returns the specified top number of rows

Sort

Sorts the incoming rows

Stream Aggregate

Groups rows by one or more columns and calculates one or more aggregate expressions



© Copyright Microsoft Corporation. All rights reserved.

The **Nested Loops** operator performs the inner join, left outer join, left semi join, and left anti semi join logical operations. Nested loops joins perform a search on the inner table for each row of the outer table, typically using an index.

The **Hash Match** operator builds a hash table by computing a hash value for each row from its build input. A `HASH()` predicate with a list of columns used to create a hash value appears in the **Argument** column. Then, for each probe row (as applicable), it computes a hash value (using the same hash function) and looks in the hash table for matches.

The **Top** operator scans the input, returning only the first specified number or percent of rows, possibly based on a sort order.

The **Sort** operator sorts all incoming rows. The **Argument** column contains either a `DISTINCT ORDER BY()` predicate if duplicates are removed by this operation, or an `ORDER BY()` predicate with a comma-separated list of the columns being sorted.

The **Stream Aggregate** operator groups rows by one or more columns and then calculates one or more aggregate expressions returned by the query. The output of this operator can be referenced by later operators in the query, returned to the client, or both.

Demo: Reading an execution plan

Generate a query execution plan
Reading a query execution plan

© Copyright Microsoft Corporation. All rights reserved.

In this demo using the AdventureWorks2017 database, run the following query and discuss its execution plan.

```
SELECT P.Name, Sum(SOD.LineTotal) as TotalSales ,SOH.OrderDate
FROM Production.Product P
INNER JOIN Sales.SalesOrderDetail SOD on SOD.ProductID = P.ProductID
INNER JOIN Sales.SalesOrderHeader SOH on SOH.SalesOrderID = SOD.SalesOrderID
GROUP BY P.Name, SOH.OrderDate
ORDER BY TotalSales DESC
```

Dynamic management objects



Dynamic Management Views and Functions supply data about database performance and state



These objects may be scoped at the database or server level



Azure SQL Database has some of its own DMVs to provide Azure specific data at the database level



All objects are in the sys schema and following the naming convention *sys.dm_**

© Copyright Microsoft Corporation. All rights reserved.

DMV:s

1) sys.dm_db_resource_stats

Varför bra? Ger snabb överblick över resursförbrukning i databasen (CPU, IO, logg, minne). Perfekt för att se om du "slår i taket".

```
SELECT TOP (10) *
FROM sys.dm_db_resource_stats
ORDER BY end_time DESC;
```

2) sys.dm_exec_requests

Varför bra? Visar vad som körs just nu: väntetyper, CPU-tid, elapsed time, blocking m.m. Superbra vid "det går långsamt NU".

```
SELECT
    session_id, status, command,
    wait_type, wait_time, blocking_session_id,
    cpu_time, total_elapsed_time
FROM sys.dm_exec_requests
WHERE session_id <> @@SPID;
```

3) sys.dm_db_index_physical_stats

Varför bra? Ger info om **fragmentering** och indexhälsa. Bra när du vill veta om indexunderhåll behövs.

```
SELECT *
FROM sys.dm_db_index_physical_stats
```

```
(
    DB_ID(), NULL, NULL, NULL, 'SAMPLED'
);
```

Vill du att jag även tar med ett exempel som visar **Top queries** (t.ex. via sys.dm_exec_query_stats)?

Dynamic management views and functions return server state information that can be used to monitor the health of a server instance, diagnose problems, and tune performance.

There are two types of dynamic management views and functions:

Server-scoped dynamic management views and functions. These require VIEW SERVER STATE permission on the server.

Database-scoped dynamic management views and functions. These require VIEW DATABASE STATE permission on the database.

Dynamic management views can be referenced in Transact-SQL statements by using two-part, three-part, or four-part names. Dynamic management functions on the other hand can be referenced in Transact-SQL statements by using either two-part or three-part names. Dynamic management views and functions cannot be referenced in Transact-SQL statements by using one-part names.

All dynamic management views and functions exist in the sys schema and follow this naming convention dm_*. When you use a dynamic management view or function, you must prefix the name of the view or function by using the sys schema.

Query Store

Introduced in SQL Server 2016, this data collection tool tracks query execution plans and runtime statistics

Can help you quickly identify queries that have regressed in performance

Allows you to easily identify the most expensive queries in your database

Available in Azure SQL Database and Azure SQL Managed Instances

A version of the Query Store is available in Azure Database for MySQL and Azure Database for PostgreSQL

© Copyright Microsoft Corporation. All rights reserved.

The Query Store can be thought of as a flight data recorder for SQL Server. It collects and more importantly permanently stores and aggregates performance information, that in older versions of SQL Server was either transient or not collected at all. The Query Store captures run time information such as duration, logical I/O, CPU usage amongst other metrics for query executions in a specific user database. It also captures the estimated execution plan for each execution, which can allow you to quickly detect an execution plan that has regressed in performance.

Query Store cont'd

Common scenarios

- Detecting regressed queries
- Determining the number of times a query was executed in a given time window
- Comparing the average execution time of a query across time windows to see large deltas

Enabled by default for new Azure SQL Database databases, not enabled for

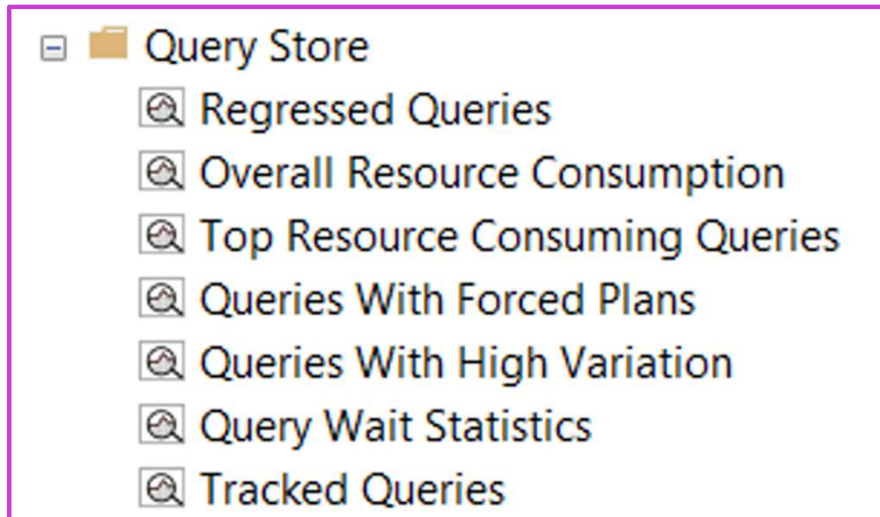
```
ALTER DATABASE <dbname> SET QUERY_STORE = ON (OPERATION_MODE = READ_WRITE);
```

© Copyright Microsoft Corporation. All rights reserved.

The Query Store can be thought of as a flight data recorder for SQL Server. It collects and more importantly permanently stores and aggregates performance information, that in older versions of SQL Server was either transient or not collected at all. The Query Store captures run time information such as duration, logical I/O, CPU usage amongst other metrics for query executions in a specific user database. It also captures the estimated execution plan for each execution, which can allow you to quickly detect an execution plan that has regressed in performance.

Query Store reports

- Several reports built into SQL Server Management Studio
- Reports can be filtered by a range of time, allowing you to identify performance issues that happened in the past
- Reports help you identify problematic changes in execution plans leading to a performance regression



© Copyright Microsoft Corporation. All rights reserved.

Regressed Queries – Hittar frågor som blivit långsammare jämfört med tidigare perioder.

Overall Resource Consumption – Visar total CPU, IO och minnesförbrukning över vald tidsperiod.

Top Resource Consuming Queries – Identifierar vilka frågor som förbrukar mest resurser totalt.

Queries With Forced Plans – Visar frågor där ett specifikt exekveringsplan har tvingats.

Queries With High Variation – Hittar frågor med stor variation i duration, CPU eller IO.

Query Wait Statistics – Visar vilka wait-typer som påverkar frågor mest över tid.

Tracked Queries – Visar särskilt markerade frågor som du valt att följa.

Regressed Queries—This shows any queries that have regressed where execution metrics have degraded in the period of time of interest (last hour, day, and week). This is a useful report for evaluating the impact of minor or major changes in server configuration or database schema.

Overall Resource Consumption—This report allows you to quickly observe the most impactful queries in your database. This allows click through to the Top Resource Consuming Query report, which allows you to gather execution plan information.

Top Resource Consuming Queries—This report shows the query and query plan, for the most impactful queries in a database for a time period. This allows you to observe if a query has multiple execution plans, and whether or not those plans have high variability in performance.

Queries with Forced Plans—This report contains information about plan forcing, and any plan forcing failures (a situation where a forced execution plan was not honored).

Queries with High Variation—This report showcases queries that have a high degree of variance between executions and can be sorted by any of the above execution metrics.

Query Wait Statistics—This report allows you to see waits statistics aggregated, and drill-through to get further information on the what queries spent the most time waiting. You should note this wait information is aggregated and not as detailed as what you might observe in the *sys.dm_os_wait_stats* DMV.

Tracked Queries—This report is filtered by query_id and allows you to view the performance of a specific query and its execution plans. You can manually enter a query_id or you can add queries from the regressed or top resource consuming query reports. The query_id can be captured from the catalog view

sys.query_store_query.

Performance overhead of Query Store



Any monitoring system has some degree of observer overhead to capture data



The query store has been developed with performance in mind and has relatively low overhead



The query store allows adjusting of collection parameters to reduce data collection on particularly busy systems

© Copyright Microsoft Corporation. All rights reserved.

Collecting data in any system has an execution cost of CPU cycles, memory, and disk utilization that is known as observer overhead. The Query Store is designed to minimize the impact of its data collection. Data for the Query Store is written to memory for each new query, and each execution of an existing query. If this was written to disk for each execution, the performance overhead would be significant, so SQL Server uses a setting called **DATA_FLUSH_INTERVAL_SECONDS** to control the frequency of flushing the Query Store data to disk. By default, the data is flushed every fifteen minutes, but this is a user configurable setting per database.

Plan forcing in the Query Store

- ▶ This feature allows you to force the database engine to use a specific execution plan for a given query
- ▶ Plan forcing is typically used as a temporary fix for a sudden query plan regression
- ▶ The full fix is to fix the query and/or indexes to ensure plan stability
- ▶ The Automatic Tuning feature in Azure SQL uses this feature

© Copyright Microsoft Corporation. All rights reserved.

Plan forcing i Query Store innebär att du kan tvinga SQL Server att använda ett specifikt exekveringsplan för en viss fråga. Det är användbart när en fråga plötsligt blir långsam, ofta efter att SQL Server valt ett nytt (sämre) plan – en så kallad *plan regression*. Då kan man snabbt återgå till ett tidigare stabilt och snabbt plan och därmed få tillbaka prestandan direkt. Plan forcing ses dock som en **tillfällig lösning**; den riktiga åtgärden är att förbättra frågan och/eller index så att SQL Server naturligt väljer rätt plan. I Azure SQL används detta även av funktionen **Automatic Tuning**, som automatiskt kan forcera bättre planer.

Another benefit of the Query Store is the ability to force a given execution plan for a query. Plan forcing in the Query Store also drives the automatic tuning feature that was introduced in SQL Server 2017, which uses the last known good execution plan after a performance regression occurs for a given query. Since the query can store multiple execution plans for a given query, you can have the database engine force a known good plan for a given query. Plan forcing should be used for queries that have suddenly changes execution plans and have experienced significant regression in execution time. Plan forcing offers a quick mitigation for a performance problem, but you should always investigate what caused the performance regression and look to resolve the cause of the execution plan variability. Those fixes could be adding an index or looking to rewrite part of a query.

Demo: Query Store

**Showcase query store reports in SQL
Server Management Studio**

© Copyright Microsoft Corporation. All rights reserved.

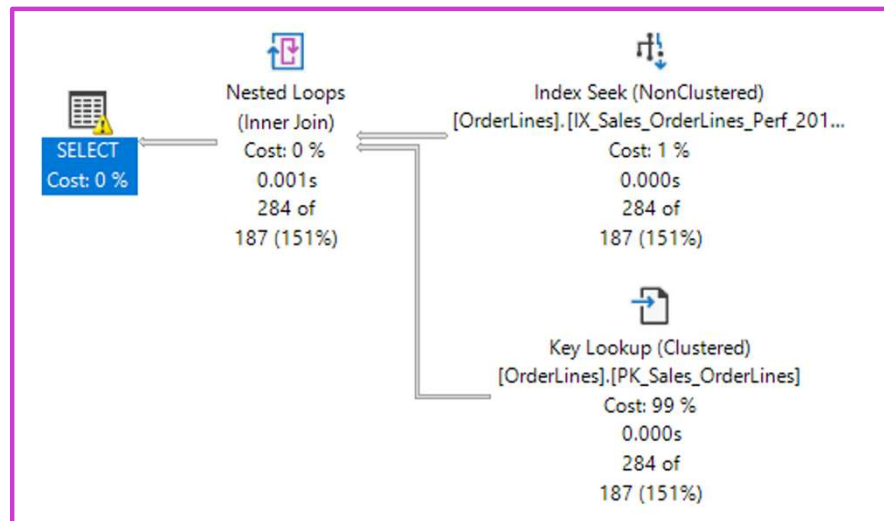
Demo Setup: Ensure the Query Store is enabled for your database. Run a number of queries against a database in Azure SQL Database or SQL Server Database

Demo: Navigate to Query Store.

Identifying problematic execution plans

This query has a few problems:

- There is a warning that can be identified by the yellow triangle on the SELECT operation
- There is also an expensive Key Lookup that is using 99.6% of the query's execution cost



© Copyright Microsoft Corporation. All rights reserved.

This query has a few problems – first there is a warning that can be identified by the yellow triangle on the SELECT operation:

This could be an implicit conversion, a spill of memory to TempDB or other concerning conditions.

There is also an expensive Key Lookup that is using 99.6% of the query's execution cost.

You would evaluate the output columns of the Index Seek operator to make decisions on potentially adding columns or creating a new index.

Easy things to look at

- Indexes – Are they being used, the number of scans versus seeks
- Sorts is it really needed – extra overhead
- Key lookups are expensive operations because they tend to happen for each row returned– having to go back to clustered index just to get extra columns that are not included in index

SARGability



This is a term that refers to a Search Arguable query predicate that can use an index to improve the performance of a query

Predicates that are not SARGs include some of the following examples:

```
WHERE LastName LIKE '%SMITH%'
```

```
WHERE CONVERT(CHAR(10), CreateDate, 121) = '2020-03-22'
```

Functions can influence the SARGability of predicates, so you should be careful with their use

© Copyright Microsoft Corporation. All rights reserved.

The term SARGable in relational databases refers to a predicate (WHERE clause) in a specific format that can leverage an index to speed up execution of a query. Predicates in the correct format are called 'Search Arguments' or SARGs. In the case of SQL Server, using a SARG means that the optimizer will evaluate using a nonclustered index on the column referenced in the SARG for a SEEK operation, instead of scanning the entire index (or the entire table) to retrieve a value.

Note that the presence of SARG does not guarantee the use of an index for a SEEK. The optimizer's costing algorithms could still determine that the index was too expensive. This could be the case if a SARG refers to a very large percentage of rows in a table. The absence of a SARG does mean that the optimizer will not even evaluate a SEEK on a nonclustered index.

Missing indexes



SQL Server includes missing index warnings in execution plans



Look for queries that may **scan a clustered index** to return a **small** amount of records from a **large table**



SQL Server stores missing index information and index utilization information in the database (***sys.dm_db_missing_index_details***)



Indexes can **impact write** performance and **consume space**, yet the performance gains offset the additional resource costs many times over



Drop an index that is not used by any queries in the database

© Copyright Microsoft Corporation. All rights reserved.

The most common performance problems we see as database administrators are due to a lack of useful indexes causing the engine to read far more pages than necessary to return the results of a query. While indexes are not free in terms of resources (adding additional indexes to a table can impact write performance and consume space) the performance gains they offer can offset the additional resource costs many times over.

The names and details of the indexes which have been deemed potentially useful are available through a dynamic management view called `sys.dm_db_missing_index_details`. There are also other DMVs in SQL Server like `sys.dm_db_index_usage_stats` and `sys.dm_db_index_operational_stats` which highlight the utilization of existing indexes. It may make sense to drop an index that is not used by any queries in the database. The missing index DMVs and plan warnings should only be used as a starting point for tuning your queries—it's important to understand what your key queries are and build indexes to support those queries. Creating all missing indexes without evaluating indexes in the context of each other is not recommended

Hardware constraints



Typically manifested through **wait statistics**



SOS_SCHEDULER_YIELD and CXPACKET wait types can be associated with **CPU pressure**, but can also point to **improper parallelism** settings or **missing indexes**



PAGEIOLATCH_* wait types can be indicative of **storage performance issues**



Detect CPU contention with '% Processor Time'



SQL Server will write to its error log if disk I/O takes longer than 15 seconds to complete



Storage system performance is best tracked at the operating system level with '**Disk Seconds/Read**' and '**Disk Seconds/Write**'

© Copyright Microsoft Corporation. All rights reserved.

WITH Waits AS

(

SELECT

ws.wait_type,

ws.wait_time_ms,

ws.signal_wait_time_ms,

ws.waiting_tasks_count,

ws.wait_time_ms - ws.signal_wait_time_ms AS resource_wait_time_ms

FROM sys.dm_os_wait_stats AS ws

WHERE ws.wait_time_ms > 0

AND ws.wait_type NOT IN

(

-- "Idle / irrelevant" waits som nästan alltid kan ignoreras

'BROKER_EVENTHANDLER', 'BROKER_RECEIVE_WAITFOR', 'BROKER_TASK_STOP',

'BROKER_TO_FLUSH', 'BROKER_TRANSMITTER',

'CHECKPOINT_QUEUE', 'CLR_AUTO_EVENT', 'CLR_MANUAL_EVENT', 'CLR_SEMAPHORE',

'DBMIRROR_DBM_EVENT', 'DBMIRROR_EVENTS_QUEUE', 'DBMIRROR_WORKER_QUEUE',

'DBMIRRORING_CMD', 'DIRTY_PAGE_POLL',

'DISPATCHER_QUEUE_SEMAPHORE',

'FT_IFTS_SCHEDULER_IDLE_WAIT', 'FT_IFTSHC_MUTEX',

'HADR_CLUSAPI_CALL', 'HADR_FILESTREAM_IOMGR_IOCOMPLETION',

'HADR_LOGCAPTURE_WAIT', 'HADR_NOTIFICATION_DEQUEUE',

'HADR_TIMER_TASK', 'HADR_WORK_QUEUE',

'LAZYWRITER_SLEEP', 'LOGMGR_QUEUE', 'MEMORY_ALLOCATION_EXT',

'ONDEMAND_TASK_QUEUE', 'PREEMPTIVE_OS_GETPROCADDRESS',

```

'PREEMPTIVE_OS_AUTHENTICATIONOPS',
'REQUEST_FOR_DEADLOCK_SEARCH',
'RESOURCE_QUEUE', 'SERVER_IDLE_CHECK', 'SLEEP_BPOOL_FLUSH',
'SLEEP_DBSTARTUP', 'SLEEP_DCOMSTARTUP', 'SLEEP_MASTERDBREADY',
'SLEEP_MASTERMDREADY', 'SLEEP_MASTERUPGRADED', 'SLEEP_MSDBSTARTUP',
'SLEEP_SYSTEMTASK', 'SLEEP_TASK', 'SLEEP_TEMPDBSTARTUP',
'SNI_HTTP_ACCEPT', 'SP_SERVER_DIAGNOSTICS_SLEEP',
'SQLTRACE_BUFFER_FLUSH', 'SQLTRACE_INCREMENTAL_FLUSH_SLEEP',
'SQLTRACE_WAIT_ENTRIES',
'WAITFOR', 'WAITFOR_TASKSHUTDOWN', 'WAIT_XTP_HOST_WAIT',
'XE_DISPATCHER_WAIT', 'XE_TIMER_EVENT'
)
)
SELECT TOP (25)
    wait_type,
    wait_time_ms,
    resource_wait_time_ms,
    signal_wait_time_ms,
    waiting_tasks_count,
    CAST(100.0 * wait_time_ms / SUM(wait_time_ms) OVER() AS decimal(6,2)) AS pct_total,
    CAST(wait_time_ms * 1.0 / NULLIF(waiting_tasks_count,0) AS decimal(12,2)) AS avg_wait_ms
FROM Waits
ORDER BY wait_time_ms DESC;

```

For the most part, hardware constraints will not manifest themselves with single query executions but will be evident when production load is applied and there is a limited number of CPU threads and a limited amount of memory to be shared among the queries. When you have CPU contention it will usually be detectable by observing the performance monitor counter '% Processor Time' which measures the CPU usage of the server. Looking deeper into SQL Server, you may see SOS_SCHEDULER_YIELD and CXPACKET wait types when the server is under CPU pressure. However, in some cases with extremely poor storage system performance, even single executions of a query that is otherwise optimized can be extremely slow. Storage system performance is best tracked at the operating system level using the performance monitor counters 'Disk Seconds/Read' and 'Disk Seconds/Write' which measure how long an I/O operation takes to complete. SQL Server will write to its error log in the event of extremely poor storage performance (if an I/O takes longer than 15 seconds to complete). If you look at wait statistics and see a high percentage of PAGEIOLATCH_SH waits in your SQL Server this can be another indication of a storage system performance issue. Typically, hardware performance is examined at a high level, early in the performance troubleshooting process, because it is relatively easy to evaluate. You can refer to the baseline information in Learning Path 4 for more information on hardware.

Statistics



Having up to date column and index statistics allows for the query optimizer to make the **best possible execution plan decisions**



SQL Server, Azure SQL Database and Managed Instance **default** to auto-updating statistics



In some cases, you may want to update statistics **more frequently** to ensure consistent performance for your queries



Use ***sys.dm_db_stats_properties*** to see the last time statistics were updated and the number of modifications that have been made since the last update



Auto-update statistics uses a dynamic percentage, which as the table grows, the percentage of row modifications that was required to trigger a statistics update gets smaller

© Copyright Microsoft Corporation. All rights reserved.

Because the query optimizer is cost-based, it is important that it has good inputs for decision making. This means that the statistics SQL Server uses to track the distribution of data in columns and indexes need be kept up to date, or it can cause suboptimal execution plans to be generated. SQL Server automatically updates its statistics as data changes in a table; however, more frequent updates may be needed for rapidly changing data. The engine uses a number of factors when building a plan including combability level of the database, row estimates based on statistics and available indexes.

Blocking and locking in SQL Server

Databases lock records and block other transactions to maintain consistency.

SQL engine uses lock escalation to minimize the impact of locks, starting with low-level locks and escalating only when necessary

Blocking activity can be problematic in two main scenarios:

- Poor transaction handling in application code (leaving transactions open for too long)
- Transactions taking too long to complete because of poor indexing or external operations like linked server queries

© Copyright Microsoft Corporation. All rights reserved.

One feature of relational databases is locking, which is essential to maintain the atomicity, consistency, and isolation properties of the ACID model. All RDBMSs will block actions that would violate the consistency and isolation of writes to a database. When programming in SQL, programmers are responsible for starting and ending transactions at the right point, in order to ensure the logical consistency of their data. In turn, the database engine provides locking mechanisms that also protect the logical consistency of the tables affected by those queries. These actions are a foundational part of the relational model.

Locking is normal behavior and happens many times during a normal day. There are two types of performance issues that can be caused by blocking:

A process holds a lock on a set of resources for an extended period of time before releasing them. This can degrade query performance and concurrency.

A process gets a lock on a set of resources, and never releases them. This problem requires administrator intervention to resolve.

Isolation levels



SQL Server offers multiple isolation levels to allow for a balance of concurrency and data quality



Isolation levels are specified at the session level, but may be overridden by the query



Lower isolation like **Read Uncommitted** may allow for more users to read the data, but increase the chance that the queries return incorrect results

© Copyright Microsoft Corporation. All rights reserved.

SQL Server offers several isolation levels to allow for a balance between concurrency and prevention of dirty reads (a scenario where second transaction reads a row being modified and sees an uncommitted value updated by the first transaction) or phantom reads (where two different sets of results are presented to the same query, because of an ongoing modification). The isolation level does not affect the locks taken to prevent data modification, a transaction will always get an exclusive lock on the data that is modifying. Lower isolation levels increase the ability of users to access data at the same time, but increase the concurrency risks that can occur. The isolation levels in SQL Server are as follows:

- **Read uncommitted**—This is the lowest isolation level available. Dirty reads are allowed, so one transaction may see changes made by another transaction that have not yet been committed.
- **Read committed**—This level allows a transaction to read data previously read, but not modified by another transaction without waiting for the first transaction to finish. This level also releases read locks as soon as the select operation is performed. This is the default SQL Server level.
- **Repeatable Read**—This level keeps read and write locks that are acquired on selected data until the end of the transaction.
- **Serializable**—This is the highest level of isolation where transactions are completely isolated. Read and write locks are acquired on selected data and not released until the end of the transaction.
- SQL Server also includes two isolation levels that include row-versioning.
- **Read Committed Snapshot**—In this alternative implementation of read committed, read operations take no row or page locks, and the engine presents each operation a snapshot of the data as it existed at the start of the operation. This level is typically used when users are running frequent reporting queries against an OLTP database, in order to prevent the read operations from blocking the write operations.
- **Snapshot**—This level provides transaction level read consistency through row versioning. This level is vulnerable to update conflicts. If a transaction running under this level reads data modified by another transaction, an update by the snapshot transaction will be terminated and roll back. This is not an issue with read committed snapshot isolation.

Monitoring for blocking problems



The *sys.dm_tran_locks* DMV contains all active locks, and can be joined with the *sys.dm_exec_requests* in order to provide statements holding locks



Querying DMVs is good for identifying a problem in real-time

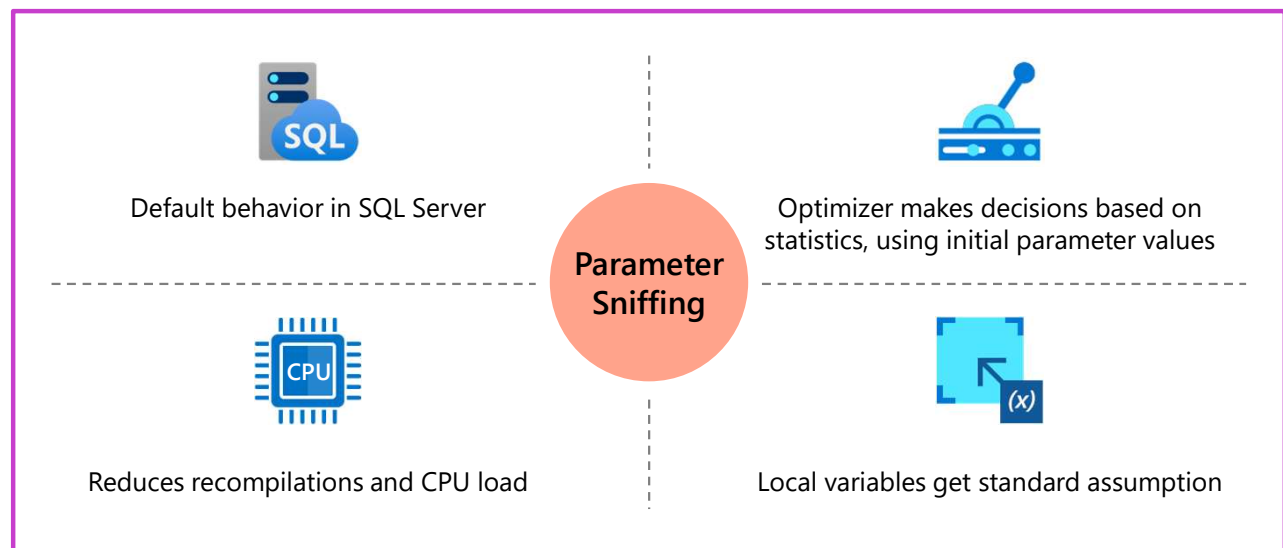


For longer term monitoring creating an extended events session to monitor for blocking issues

© Copyright Microsoft Corporation. All rights reserved.

Identifying blocking problem can be troublesome as they can be sporadic in nature. There is a DMV called *sys.dm_tran_locks*, which can be joined with *sys.dm_exec_requests* in order to provide further information on locks that each session is holding. A better way to monitor for blocking problems is to do so on an ongoing basis using the Extended Events engine. In the additional resources of this Learning Path, you can see of an example extended events session that will track locks held and correlate them with queries so you can monitor queries over time.

What is parameter sniffing?



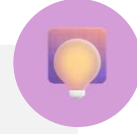
© Copyright Microsoft Corporation. All rights reserved.

Parameter sniffing betyder att SQL Server **tittar ("sniffar") på värdet** på en parameter första gången en fråga körs, och använder det för att välja **en exekveringsplan**.

Sedan **sparas planen i cache** och återanvänds för nästa körning – även om nästa parameter-värde är helt annorlunda.

Query execution plans are arguably the most important aspect of database performance. Improving bad plans is certainly an area where a small amount of effort can bring huge improvements. While hardware issues can limit query performance, improving hardware usually yields performance improvements in the 10-20% range. More commonly database administrators encounter queries that are not optimized, have stale or missing statistics, have missing indexes, or poor database design choices that lead to the database engine doing more work than is necessary to return results for a given query. Improving the plans can sometimes yield performance improvements in the 100-200% range or even more, meaning that after improving a plan with better indexes or statistics, a query could run twice or three times as fast!

Knowledge check



Which type of execution plan is stored in the plan cache?

- ☒ Estimated execution plan
- ☐ Actual execution plan
- ☐ Live Query Stats



Which DMV should you use to find index utilization?

- ☒ sys.dm_db_index_usage_stats
- ☐ sys.dm_db_missing_index_details
- ☐ sys.dm_exec_query_plan_stats



Which of the following wait types would indicate excessive CPU consumption?

- ☒ SOS_SCHEDULER_YIELD
- ☐ RESOURCE_SEMAPHORE
- ☐ PAGEIOLATCH_SH

© Copyright Microsoft Corporation. All rights reserved.

Which type of execution plan is stored in the plan cache?

- ☒ **Estimated execution plan** – Plan cache lagrar kompillerad/estimerad plan, inte faktisk kördata.
- ☒ **Actual execution plan** – Innehåller runtime-statistik; sparas normalt inte i plan cache.
- ☒ **Live Query Stats** – Är realtidsvisning, inte en plan som cachas.

Which DMV should you use to find index utilization?

- ☒ **sys.dm_db_index_usage_stats** – Visar indexanvändning: seeks, scans, lookups och uppdateringar.
- ☒ **sys.dm_db_missing_index_details** – Visar saknade indexförslag, inte faktisk indexanvändning.
- ☒ **sys.dm_exec_query_plan_stats** – Ger planstatistik, inte indexanvändning per index.

Which of the following wait types would indicate excessive CPU consumption?

- ☒ **SOS_SCHEDULER_YIELD** – Indikerar CPU-press; trådar väntar på schemaläggningstid.
- ☒ **RESOURCE_SEMAPHORE** – Handlar om minnesbrist för query workspace, inte CPU.
- ☒ **PAGEIOLATCH_SH** – Indikerar disk-I/O väntan vid läsning av datasidor.

Instructor led labs: Identify and resolve blocking issues

Run blocked queries report
Enable Read Commit Snapshot isolation level
Evaluate performance improvements

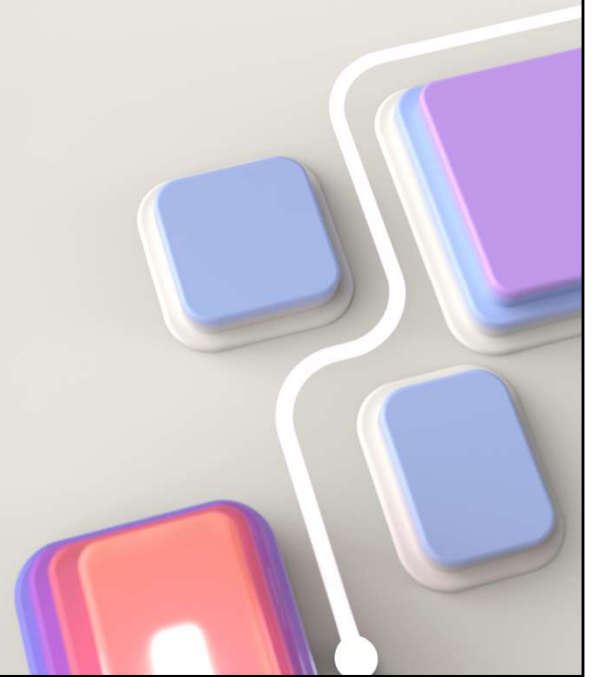
© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/08-identify-resolve-blocking-issues.html>

Explore performance-based database design

<https://learn.microsoft.com/training/modules/explore-performance-based-design/>

© Copyright Microsoft Corporation. All rights reserved.



Objectives



Explore normal forms and how they affect database design



Choose appropriate datatypes for your data



Evaluate appropriate index types

© Copyright Microsoft Corporation. All rights reserved.

Database design is an important aspect of database performance, even though it's not always under the control of the database administrator. You may be working with third- party vendor applications that you didn't build. Whenever possible, it's important to design your database properly for the workload, whether it's an online transaction processing (OLTP) or data warehouse workload. Many design decisions, such as choosing the right datatypes, can make large differences in the performance of your databases.

Normalization (First Normal Form)

Create a separate table for each set of related data

Eliminate repeating groups in individual tables

Identify each set of related data with a primary key

First Normal Form eliminates repeating groups, each row refers to a particular product, and there is a ProductID to be used as a primary key

ProductID	ProductName	Price	ProductionCountry	ShortLocation
1	Widget	15.95	United States	US
2	Foop	41.95	United Kingdom	UK
3	Glombit	49.95	United Kingdom	UK
4	Sorfin	99.99	Republic of the Philippines	RepPhil
5	Stem Bolt	29.95	United States	US

© Copyright Microsoft Corporation. All rights reserved.

First Normal Form (1NF) handlar om att göra tabeller enkla och tydliga. Grundidén är att varje kolumn ska innehålla **ett enda värde** (inte listor eller flera värden i samma cell) och att tabellen inte ska ha **upprepanade grupper** av data. Varje rad ska beskriva **en unik sak**, till exempel en produkt, och därför behövs en **primärnyckel** som identifierar raden – här används **ProductID**. I exemplet ligger produktinformation som namn och pris i samma tabell, och varje produkt har ett eget ID. På så sätt blir det lättare att söka, uppdatera och undvika felaktig eller duplicerad information.

Database normalization is a design process used to organize a given set of data into tables and columns in a database. Each table should contain data relating to a specific 'thing' and only have data that supports that same 'thing' included in the table. The goal of this process is to reduce duplicate data contained within your database, to reduce the performance impact of database inserts and updates. For example, a customer address change is much easier to implement if the only place the customer address is stored in the Customers table.

Normalization (Second Normal Form)

If a table does not have a composite key, first and second normal form are the same

Same requirements as first normal form, plus..

If the table has a composite key, all attributes must depend on the complete key, not just part of it

ProductID	ProductName	Price	ProductionCountry	ShortLocation
1	Widget	15.95	United States	US
2	Foop	41.95	United Kingdom	UK
3	Glombit	49.95	United Kingdom	UK
4	Sorfin	99.99	Republic of the Philippines	RepPhil
5	Stem Bolt	29.95	United States	US

© Copyright Microsoft Corporation. All rights reserved.

Andra normalformen (2NF) gäller tabeller med sammansatt nyckel: alla icke-nyckelkolumner måste bero på hela nyckeln, annars uppstår upprepning och uppdateringsproblem.

Andra normalformen (2NF) bygger vidare på första normalformen (1NF). Den blir egentligen bara viktig när en tabell har en **sammansatt primärnyckel** (composite key), alltså när man behöver två eller flera kolumner tillsammans för att identifiera en rad unikt. Kravet i 2NF är att **alla kolumner i tabellen måste bero på hela nyckeln**, inte bara en del av den. Om en kolumn bara beror på ena delen av nyckeln får man onödigt upprepning av data, vilket kan skapa problem vid uppdateringar. Har tabellen **ingen sammansatt nyckel**, är 1NF och 2NF i praktiken samma sak.

Second normal form is only relevant to tables with composite keys, like ProductColor in Table 2 above. Consider the case where the ProductColor table also includes the product's price. This table has a composite key on ProductID and Color, because only using both column values can we uniquely identify a row. If a product's price does not change with the color, we might see data as shown in Table 4.

The example shown on this slide is not in Second Normal Form. The price value is dependent on the ProductID but not on the Color. There are three rows for ProductID 1, so the price for that product is repeated 3 times. The problem with violating second normal form is that if we have to update the price, we have to make sure we update it everywhere. If we update the price in the first row, but not the second or third, we would have something called an 'update anomaly'. After the update, we wouldn't be able to tell what the actual price for ProductID 1 was. The solution is to move the Price column to a table that has ProductID as a single column key, because that is the only column that Price depends on. For example, we could use Table 3 to store the Price.

Normalization (Third Normal Form)

Same requirements as second normal form, plus..

All non-key relationships are non-transitively dependent on the primary key

Most OLTP databases should be in third normal form or as it is commonly abbreviated 3NF

ProductID	ProductName	Price	ProductionCountry	ShortLocation
1	Widget	15.95	United States	US
2	Foop	41.95	United Kingdom	UK
3	Glombit	49.95	United Kingdom	UK
4	Sorfin	99.99	Republic of the Philippines	RepPhil
5	Stem Bolt	29.95	United States	US

This table is NOT in 3rd normal form because ShortLocation depends on ProductionCountry, and not just on the primary key

© Copyright Microsoft Corporation. All rights reserved.

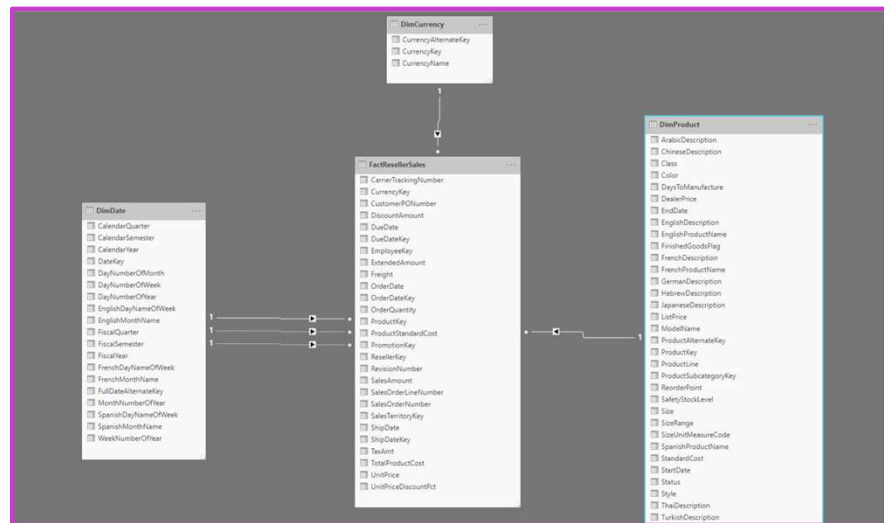
Tredje normalformen (3NF) kräver att icke-nyckelkolumner bara beror på primärnyckeln, inte på andra icke-nyckelkolumner. Detta minskar redundans och undviker uppdateringsanomalier.

Second normal form is only relevant to tables with composite keys, like ProductColor in Table 2 above. Consider the case where the ProductColor table also includes the product's price. This table has a composite key on ProductID and Color, because only using both column values can we uniquely identify a row. If a product's price does not change with the color, we might see data as shown in Table 4.

The example shown on this slide is not in Second Normal Form. The price value is dependent on the ProductID but not on the Color. There are three rows for ProductID 1, so the price for that product is repeated 3 times. The problem with violating second normal form is that if we have to update the price, we have to make sure we update it everywhere. If we update the price in the first row, but not the second or third, we would have something called an 'update anomaly'. After the update, we wouldn't be able to tell what the actual price for ProductID 1 was. The solution is to move the Price column to a table that has ProductID as a single column key, because that is the only column that Price depends on. For example, we could use Table 3 to store the Price.

Denormalization

- Denormalized database designs are commonly used in data warehouses and data mart
- This approach reduces the number of joins needed for reporting but is less effective for updates.
- Two examples of denormalized structures are star (shown below) and snowflake schemas



© Copyright Microsoft Corporation. All rights reserved.

While the 3rd normal form is theoretically desirable, it is not always possible for all data. In addition, a completely normalized database does not always give you the best performance. Normalized data frequently requires multiple join operations to get all the necessary data returned in a single query. There is a tradeoff between normalizing data where the number of joins required to return query results has high CPU utilization, and denormalized data that has fewer joins and less CPU required, but opens up the possibility of update anomalies.

Note that denormalized data is not the same as unnormalized. For denormalization, we start by designing tables that are normalized. Then we can add additional columns to some tables to reduce the number of joins required, but as we do so, we are aware of the possible update anomalies. We then make sure we have triggers or other kinds of processing that will make sure that when we perform an update, all the duplicate data is also updated.

Understand data types for performance

- ▶ Importantly, ensure that the data types in your application code match the data types in your database tables
- ▶ Conversions are expensive, and can cause much more expensive execution plans
- ▶ In general, smaller data types are better where possible
 - ▶ Data compression can help with this

© Copyright Microsoft Corporation. All rights reserved.

Unicode data

Normal data types use **1 byte** per character

The problem with this is that the data type can only represent about **256 characters**

Unicode types use 2 bytes per character and can support up to **65,536 patterns**

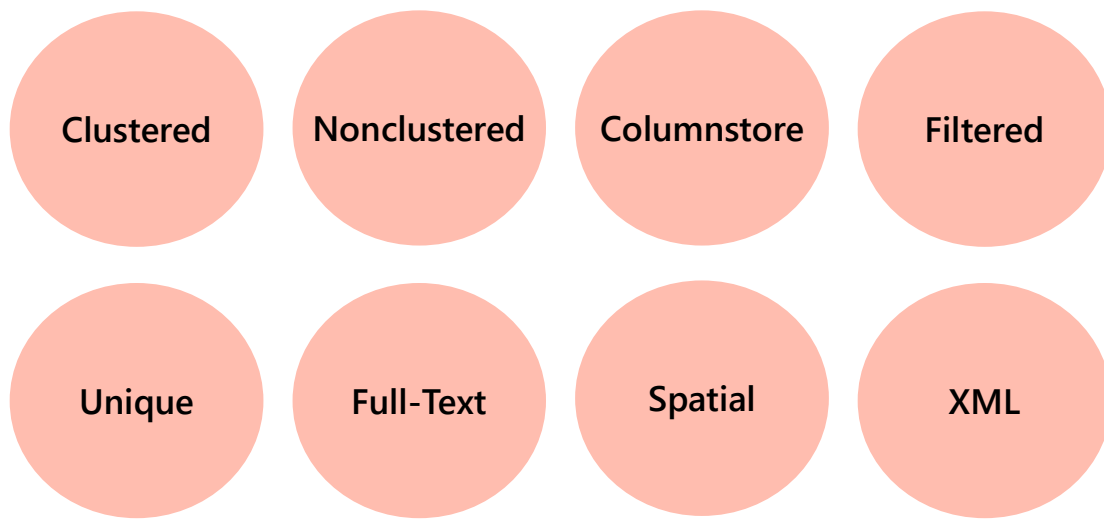
Unicode data is needed for Cyrillic languages, Asian languages and many symbols

© Copyright Microsoft Corporation. All rights reserved.

One problem with data types that use 1 byte to encode each character is that the data type can only represent 256 different characters. This forces multiple encoding specifications, or code pages, for different alphabets such as European alphabets, which are relatively small. It is also impossible to handle systems such as the Japanese Kanji or Korean Hangul alphabets that have thousands of characters.

Each Microsoft SQL Server collation has a code page that defines what patterns of bits represent each character in **char**, **varchar**, and **text** values. Individual columns and character constants can be assigned a different code page. Client computers use the code page associated with the operating system locale to interpret character bit patterns. There are many different code pages and some characters appear on some code pages, but not on others. Some characters are defined with one bit pattern on some code pages, and with a different bit pattern on other code pages. When you build international systems that must handle different languages, it becomes difficult to pick code pages for all the computers that meet the language requirements of multiple countries and regions. It is also difficult to ensure that every computer performs the correct translations when interfacing with a system using a different code page.

Types of indexes



© Copyright Microsoft Corporation. All rights reserved.

1.Clustered Index: Sorts and stores the data rows in the table based on the index key. Each table can have only one clustered index.

2.Nonclustered Index: Contains a separate structure from the data rows, with pointers to the data. A table can have multiple non-clustered indexes.

3.Unique Index: Ensures that the index key contains unique values, preventing duplicate entries.

4.Filtered Index: A non-clustered index with a WHERE clause to index a subset of rows.

5.Full-Text Index: Supports full-text queries against character-based data.

6.Spatial Index: Optimizes queries for spatial data types like geometry and geography.

7.XML Index: Optimizes queries on XML data stored in XML columns.

8.Columnstore Index: Stores data in a columnar format, ideal for data warehousing and analytics.

Designing indexes



Cluster key values can help minimize page splits and improve data locality and help to decrease disk seek time



Clustered Indexes should be as narrow (minimal columns) as possible



In nonclustered index keys, the most selective column should be the leftmost column in the key



Use included columns in your nonclustered indexes where possible, to keep keys narrower



You should not create redundant indexes and avoid creating unused indexes on your tables

© Copyright Microsoft Corporation. All rights reserved.

SQL Server has several index types to support different types of workloads. At a high level, an index can be thought of as an on-disk structure that is associated with a table or a view, that enables SQL Server to more easily find the row or rows associated with the index key (which consists of one or more columns in the table or view), compared to scanning the entire table

Clustered indexes



A clustered index sorts and stores the data in a table based on key values



There can only be one clustered index per table, since rows can only be stored in one order



Clustered indexes are frequently the primary key for a table



Clustered indexes have no uniqueness requirement

© Copyright Microsoft Corporation. All rights reserved.

A common DBA job interview question is to ask the candidate the difference between a clustered and nonclustered index—as indexes are a fundamental data storage technology in SQL Server. A clustered index is the underlying table, stored in sorted order based on the key value. There can only be one clustered index on a given table, because the rows can be stored in one order. A table without a clustered index is called a heap, and heaps are typically only used as staging tables.

An important design principle is to keep your clustered index key as narrow as possible. When considering the key column(s) for your clustered index you should consider columns that are unique or that contain many distinct values. Another property of a good clustered index key is for records that are accessed sequentially, and are used frequently to sort the data retrieved from the table. Having the clustered index on the column used for sorting can prevent the cost of sorting every time that query executes, because the data will be already stored in the desired order.

Nonclustered indexes



Nonclustered indexes are secondary indexes used to help the performance of queries not served by the clustered index



You can create multiple nonclustered indexes on a table



You can also create filtered indexes, for tables with large data skew



The cost of nonclustered indexes is space and data modification operations

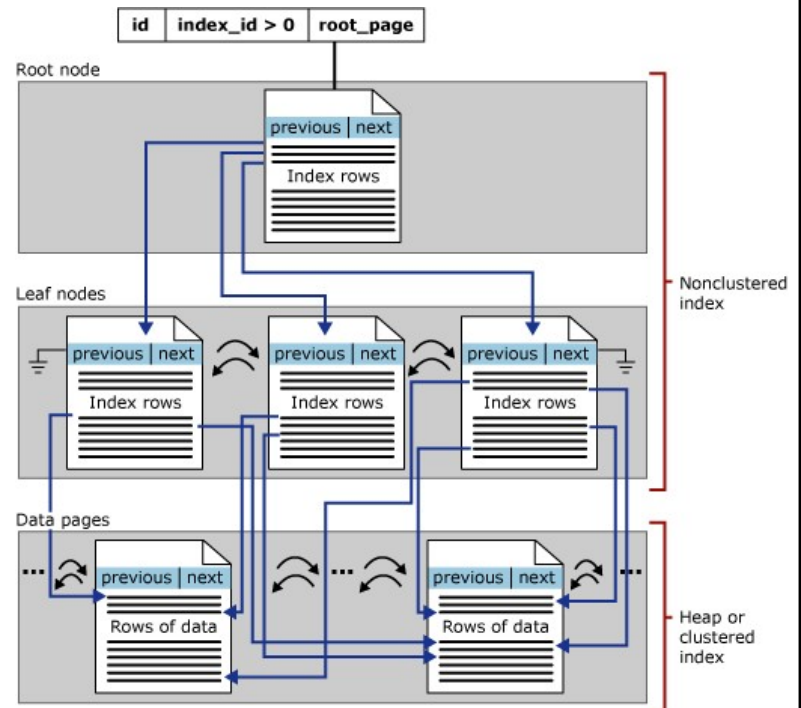
© Copyright Microsoft Corporation. All rights reserved.

Nonclustered indexes are a separate structure from the data rows. A nonclustered index contains the key values defined for the index, and a pointer to the data row that contains the key value. You can add additional nonkey columns to the leaf level of the nonclustered index to cover more columns using the included columns feature in SQL Server. You can create multiple nonclustered indexes on a table.

How nonclustered indexes work

- Each page in an index b-tree is called an index node, and the top node of b-tree is called the root node.
- The bottom nodes in an index are called leaf nodes and the collection of leaf nodes is the leaf level.

© Copyright Microsoft Corporation. All rights reserved.



Each page in an index b-tree is called an index node, and the top node of b-tree is called the root node. The bottom nodes in an index are called leaf nodes and the collection of leaf nodes is the leaf level.

Index design is a mixture of art and science. A narrow index with few columns in its key requires less time to update and has lower maintenance overhead; however it may not be useful for as many queries as a wider index that includes more columns. You may need to experiment with several indexing approaches based on the columns selected by your application's queries. The query optimizer will generally choose what it considers to be the best existing index for a query; however, that doesn't mean that there isn't a better index that could be built.

Columnstore indexes



Data is stored in columns, not rows



This allows for higher levels of compression, which reduces storage and memory footprint of data



Columns that are not referenced in a query are not scanned, giving reducing the amount of I/O needed



Best used in large tables (e.g., data warehouse fact tables, temporal history tables)



Clustered columnstore must include all columns of the table



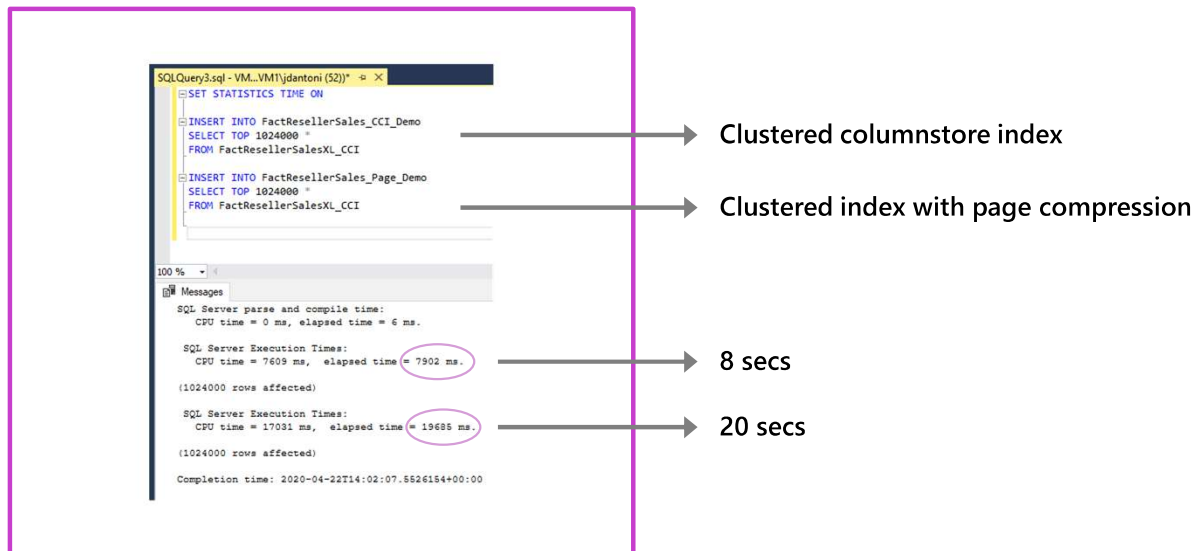
Use **clustered columnstore index** for fact tables and large dimension tables for DW workloads. Use **nonclustered columnstore index** to perform analysis in real time on an OLTP workloads

© Copyright Microsoft Corporation. All rights reserved.

Questions to ask around columnstore indexes

- Is your table large enough?
- Is your data volatile, meaning changing frequently?
- What datatypes are in your table?
- Are you doing aggregations or performing analytics on the data or are you looking for specific values?
- Is this a data warehouse fact or dimension table?

Columnstore indexes – Data load performance comparison

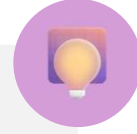


© Copyright Microsoft Corporation. All rights reserved.

The other benefit columnstore indexes offer to data warehouse workloads is an optimized load path for bulk insert operations of 102,400 rows or more. While 102,400 is the minimum value to load directly into the columnstore, each collection of rows, called a rowgroup, can be up to approximately 1,024,000 rows.

Having fewer, but fuller, rowgroups makes your SELECT queries more efficient, because fewer row groups need to be scanned to retrieve the requested records. These loads take place in memory and are directly loaded to the index. For smaller volumes, data is written to a b-tree structure called a delta store, and asynchronously loaded into the index.

Knowledge check



What type of database design should you use for a data warehouse when you want to reduce the data volume of your dimensions?

- ☐ Star Schema
- ☐ 3rd normal form
- ☒ Snowflake schema



What is the minimum number rows do you need to bulk insert into a columnstore index?

- ☐ 1,000,000
- ☒ 102,400
- ☐ 1000



Which compression type offers the highest level of compression?

- ☐ Page Compression
- ☐ Row Compression
- ☒ Columnstore Archival

© Copyright Microsoft Corporation. All rights reserved.

1) What type of database design should you use for a data warehouse when you want to reduce the data volume of your dimensions?

- ☒ **Correct: Snowflake schema** – normalizes dimensions into smaller tables, less duplicated data.
- ☒ **Star Schema** – denormalized dimensions; simpler queries but more duplicated dimension data.
- ☒ **3rd normal form** – OLTP-style design, not typical dimensional warehouse modelling.

2) What is the minimum number rows do you need to bulk insert into a columnstore index?

- ☒ **Correct: 102,400** – minimum rows for a compressed rowgroup in columnstore.
- ☒ **1,000,000** – common misconception; not required for columnstore compression.
- ☒ **1000** – far too low; will end up in deltastore, not compressed.

3) Which compression type offers the highest level of compression?

- ☒ **Correct: Columnstore Archival** – highest compression ratio, best for cold data.
- ☒ **Page Compression** – strong rowstore compression, but not as high as archival.
- ☒ **Row Compression** – lowest of these; mostly datatype/storage optimization.

Instructor led labs: Identify database design issues

Examine the query and identify the problem
Identify ways to fix the warning message
Improve the code

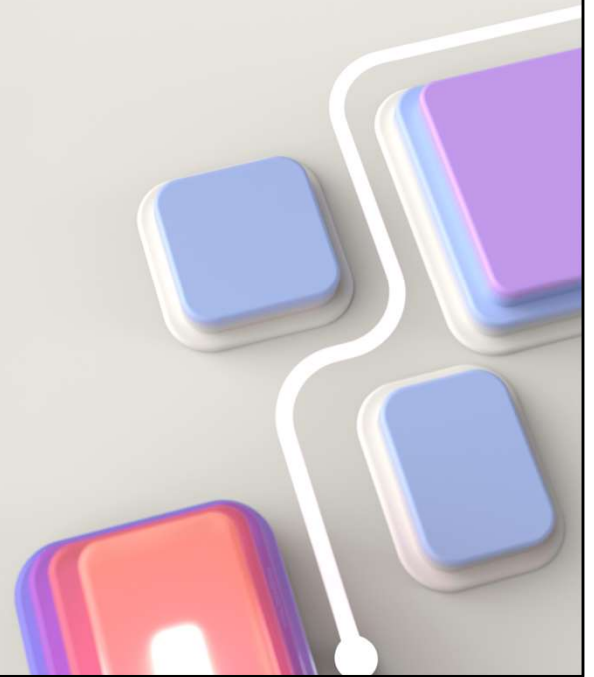
© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/09-identify-issues-database-design.html>

Evaluate performance improvements

<https://learn.microsoft.com/training/modules/evaluate-performance-improvements/>

© Copyright Microsoft Corporation. All rights reserved.



Objectives



Describe wait statistics



Understand important aspects when tuning indexes



Explore when to use query hints

© Copyright Microsoft Corporation. All rights reserved.

One of the challenges of the DBA's role is to evaluate the changes they make to code or data structures on a busy, production system. While tuning a single query in isolation offers easy metrics such as elapsed time or logical reads, making minor tweaks on a busy system may require deeper evaluation.

Wait statistics



Analysis of wait statistics is a holistic method of evaluating server performance



The database engine tracks what each thread is waiting on



This wait data is logged into memory and then persisted to disks



This can be helpful to isolate hardware, code, and configuration problems



Stored in *sys.dm_os_waits_stats* and in *sys.dm_db_wait_stats* in Azure SQL Database

© Copyright Microsoft Corporation. All rights reserved.

One holistic way of monitoring server performance is to evaluate what the server is waiting on—SQL Server is instrumented with a wait tracking system, which monitors each running thread and logs what resources the thread is waiting on. This data is aggregated in the DMV *sys.dm_os_wait_stats* (and in *sys.dm_db_wait_stats* in Azure SQL Database) and is also tracked for active sessions in *sys.dm_exec_session_wait_stats*. These statistics (SQL Server tracks over 900 wait types) allow the DBA to get an overview of the performance of the server, and to readily identify configuration or hardware issues. This data is generally persisted from the time of instance startup, but the data can be flushed as needed to observe changes.

SQL Server tracks **over 900** wait types

- Wait statistics are evaluated as a percentage of the total waits on the server
1. **Resource waits** occur when a worker thread requests access to a resource that is currently being used - **locks, latches, and disk I/O waits**.
 2. **Queue waits** occur when a worker thread is idle - **deadlock**
 3. **External waits** occur when waiting on an external process like a **linked server query** to complete, or a **network wait** related to returning a large result set.
 - Data is aggregated in the DMV **sys.dm_os_wait_stats** (and in **sys.dm_db_wait_stats** in Azure SQL Database) and tracked for active sessions in **sys.dm_exec_session_wait_stats**
 - Data is generally persisted from the time of instance startup and cleared as needed

Examples of wait statistics

High **RESOURCE_SEMAPHORE** waits

This is indicative of **queries waiting on memory to become available**, and may indicate excessive memory grants to some queries

High **LCK_M_X** waits

This can indicate a **blocking problem** that can be solved by either changing to the READ COMMITTED SNAPSHOT isolation level, or making changes in indexing to reduce transaction times, or possibly better transaction management within T-SQL code

High **PAGEIOLATCH_SH** waits

This can indicate a **problem with indexes**, where SQL Server is scanning too much data, or if the wait count is low, but the wait time is high, can indicate storage performance problems

High **SOS_SCHEDULER_YIELD** waits

This can indicate **high CPU utilization** – Very high workloads, or missing indexes

© Copyright Microsoft Corporation. All rights reserved.

One holistic way of monitoring server performance is to evaluate what the server is waiting on—SQL Server is instrumented with a wait tracking system, which monitors each running thread and logs what resources the thread is waiting on. Wait statistics are broken down into three types of waits, resource waits, queue waits, and external waits.

Resource waits occur when a worker thread in SQL Server requests access to a resource that is currently being used by a thread. Examples resources waits are locks, latches, and disk I/O waits.

Queue waits occur when a worker thread is idle and waiting for a work to be assigned. Example queue waits are deadlock monitoring and deleted record cleanup.

External waits occur when SQL Server is waiting on a external process like a linked server query to complete. An example of an external wait is a network wait related to returning a large result set to a client application.

This data is aggregated in the DMV *sys.dm_os_wait_stats* (and in *sys.dm_db_wait_stats* in Azure SQL Database) and is also tracked for active sessions in *sys.dm_exec_session_wait_stats*. These statistics (SQL Server tracks over 900 wait types) allow the DBA to get an overview of the performance of the server, and to readily identify configuration or hardware issues. This data is generally persisted from the time of instance startup, but the data can be flushed as needed to observe changes.

Wait statistics are evaluated as a percentage of the total waits on the server a

The results of this query from *sys.dm_os_wait_stats* shows the wait type, and the aggregation of Percent of time waiting (Wait Percentage) and the average wait time in seconds for each wait type. In this case the server has Always On Availability Groups in place, as indicated by the REDO_THREAD_PENDING_WORK and PARALLEL_REDO_TRAN_TURN wait types. The relatively high percentage of CXPACKET and SOS_SCHEDULER_YIELD waits indicates that this server is under some CPU pressure.

There are many wait types that relate to specific and common SQL Server performance issues:

RESOURCE_SEMAPHORE waits—this is indicative of queries waiting on memory to become available, and may indicate excessing memory grants to some queries. This is typically observed by long query runtimes or even time outs. These waits can be caused by out of date statistics, missing indexes, and excessive query concurrency.

LCK_M_X waits—frequent occurrences of this wait type can indicate a blocking problem, that can be solved by either changing to the READ COMMITTED SNAPSHOT isolation level, or making changes in indexing to reduce transaction times, or possibly better transaction management within T-SQL code.

PAGEIOLATCH_SH waits—this wait type can indicate a problem with indexes (or a lack of useful indexes), where SQL Server is scanning too much data. Alternatively, if the wait count is low, but the wait time is high, it can indicate storage performance problems. You can observe this by analyzing the data in the **waiting_tasks_count** and the **wait_time_ms** in the *sys.dm_os_wait_stats* DMV, to calculate an average wait time for a given wait type.

SOS_SCHEDULER_YIELD waits---this wait type can indicate high CPU utilization which is generally correlated with either very high workloads, or missing indexes, often in conjunction with high levels of CXPACKET waits as shown above in Figure 17.

Demo: Query wait statistics

Querying sys.dm_os_wait_stats
Reviewing Query Store Wait report

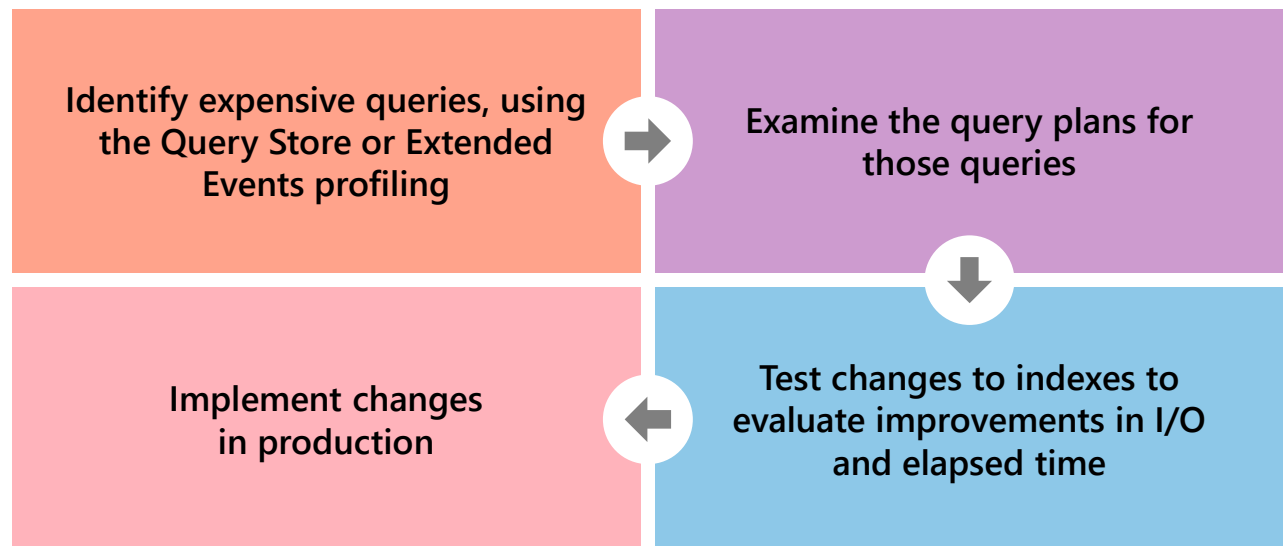
© Copyright Microsoft Corporation. All rights reserved.

Using Azure SQL Database, run the following query and discuss the results.

```
SELECT [wait_type] ,[waiting_tasks_count] ,[wait_time_ms] ,[wait_time_ms]/[waiting_tasks_count] AS  
[avg_wait_time_ms] ,[max_wait_time_ms] ,[signal_wait_time_ms] FROM sys.dm_db_wait_stats ORDER BY  
[avg_wait_time_ms] DESC
```

Demonstrate the Query Store Query Wait Statistics report.

Index tuning methodology



© Copyright Microsoft Corporation. All rights reserved.

The most common (and most effective) method to tuning T-SQL queries is to evaluate and adjust your indexing strategy. Properly indexed databases perform fewer IOs to return query results which reduces pressure on IO, storage, and even allows for better memory utilization. It is also important to keep in mind the read/write ratio of your database. A heavy write workload indicates that the cost writing records to additional indexes may not be of much benefit, unless that write workload is more update operations which do lookup, in which case additional indexes or columns added to an existing index may be beneficial. Your goal should always be to get the most benefit out of the smallest number of indexes on your tables.

Index tuning

The goal is faster queries using the fewest indexes and IOs

Common tuning approach:

Evaluate existing index usage - *sys.dm_db_index_operational_stats* and *sys.dm_db_index_usage_stats*

Eliminate unused and duplicate indexes

Review and evaluate expensive queries from the Query Store or Extended Events and refactor where appropriate

Create the index(s) in a non-production environment and test query execution and performance

Deploy the changes to production

© Copyright Microsoft Corporation. All rights reserved.

As data is inserted, updated, and deleted from indexes the logical ordering in the index does not match the physical ordered inside of the pages making up the indexes. Also, over time the data modifications can cause the data to become scattered or fragmented in the database. This fragmentation can degrade query performance because the database engine needs to read additional pages in order to locate the data to where the index points.

Index maintenance

Over time, inserts, updates and deletes can cause fragmentation in an index

Fragmentation causes the data to occupy more pages than it otherwise would

Fragmentation can cause query performance degradation

Indexes should be regularly **reorganized** or **rebuilt**

SQL Server and Azure SQL Database allow for indexes to rebuilt **online**

Newer versions of SQL Server allow **resumable** index operations

Stored in the DMV *Sys.dm_db_index_physical_stats*

© Copyright Microsoft Corporation. All rights reserved.

As data is inserted, updated, and deleted from indexes the logical ordering in the index does not match the physical ordered inside of the pages making up the indexes. Also, over time the data modifications can cause the data to become scattered or fragmented in the database. This fragmentation can degrade query performance because the database engine needs to read additional pages in order to locate the data to where the index points.

Index maintenance

Reorganization

- Defragments an index by physically reordering the leaf-level index pages to match the logical sorted order of the leaf nodes
- Compacts the index pages based on the index's fill factor setting
- It is an online process

Rebuilding

- Drops and recreates the pages of the index
- Causes the statistics to be updated
- Use when fragmentation is greater than 30%
- It can be either online or offline

© Copyright Microsoft Corporation. All rights reserved.

Bilden beskriver två vanliga sätt att underhålla index i SQL Server: **Reorganization** och **Rebuilding**. En *reorganize* defragmenterar indexet genom att fysiskt ordna om leaf-sidorna så att de bättre följer den logiska sorteringsordningen. Den komprimerar också sidorna enligt indexets **fill factor**, och processen är **online**, vilket betyder att användare kan fortsätta arbeta under tiden. En *rebuild* är mer "kraftfull": den skapar om indexet genom att i praktiken droppa och bygga upp sidorna på nytt, och samtidigt uppdateras även **statistik**. Rebuild används ofta när fragmenteringen är hög (t.ex. >30 %) och kan köras **online eller offline** beroende på edition och inställningar.

The most common (and most effective) method to tuning T-SQL queries is to evaluate and adjust your indexing strategy. Properly indexed databases perform fewer IOs to return query results which reduces pressure on IO, storage, and even allows for better memory utilization. It is also important to keep in mind the read/write ratio of your database. A heavy write workload indicates that the cost writing records to additional indexes may not be of much benefit, unless that write workload is more update operations which do lookup, in which case additional indexes or columns added to an existing index may be beneficial. Your goal should always be to get the most benefit out of the smallest number of indexes on your tables.

Columnstore index maintenance

Columnstore indexes also require maintenance due to fragmentation

If 20% or more of the rows are deleted, you should consider **reorganizing** your index

Fragmentation is caused when deletes and updates happen as they both leave deleted rows in the columnstore index

These deleted records are not included in query results, but are scanned in each query

Fragmentation is measured by looking for deleted rows in the *sys.dm_dm_colum_store_row_group_physical_stats* DMV

© Copyright Microsoft Corporation. All rights reserved.

Columnstore index fragmentation is not reported in same manner as b-tree index fragmentation. You should examine the *sys.dm_dm_colum_store_row_group_physical_stats* for deleted rows—which will be caused by both update and delete operations. A good metric to measure fragmentation in a columnstore index is based on the deleted rows in the index. If 20% or more of the rows are deleted, you should consider reorganizing your index. In a test environment, verify whether a reorganization improves the performance of the workloads using the columnstore index.

Query hints

Query hints enforce the specified behavior across the entire query

They affect all the operators in the query

Some common use cases for query hints include:

- Limit the amount of memory granted to query
- Force the use of a specific index
- Control join behavior

© Copyright Microsoft Corporation. All rights reserved.

```
USE AdventureWorks2017;
GO
```

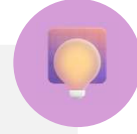
```
SELECT p.Name, p.ProductNumber
FROM Production.Product AS p WITH (INDEX(PK_Product_ProductID))
WHERE p.ProductID BETWEEN 1 AND 100;
```

```
USE AdventureWorks2017;
GO
```

```
SELECT *
FROM Sales.SalesOrderDetail
ORDER BY UnitPrice DESC
OPTION (MAX_GRANT_PERCENT = 5);
--Begränsar hur mycket minne frågan får begära för t.ex. sort/hash.
```

When you are hitting a problem in the query optimizer that you can't solve with any other options a query hint is a way of directing the optimizer what to do.

Knowledge check



What type of index is best used on a data warehouse fact table?

- ☐ Nonclustered columnstore
- ☐ Clustered b-tree
- ☒ Clustered columnstore



Which DMV provides information about server level wait statistics?

- ☐ Sys.dm_exec_session_wait_stats
- ☒ Sys.dm_os_wait_stats
- ☐ Sys.dm_db_index_physical_stats



Which DMV can you use to capture the actual execution plan for a given query?

- ☐ sys.dm_exec_query_plan
- ☐ sys.dm_exec_cached_plans
- ☒ sys.dm_exec_query_plan_stats

© Copyright Microsoft Corporation. All rights reserved.

What type of index is best used on a data warehouse fact table?

- ☒ Clustered columnstore – Ideal for large analytic queries, high compression, and batch operations.
- ☒ Nonclustered columnstore – Often used on dimensions or secondary, not main fact table.
- ☒ Clustered b-tree – Best for OLTP workloads, not suited for large-scale warehouse fact tables.

Which DMV provides information about server level wait statistics?

- ☒ Sys.dm_os_wait_stats – Shows aggregated wait statistics across the entire SQL Server instance.
- ☒ Sys.dm_exec_session_wait_stats – Provides waits at a session level, not overall server.
- ☒ Sys.dm_db_index_physical_stats – Gives fragmentation and index stats, not wait information.

Which DMV can you use to capture the actual execution plan for a given query?

- ☒ Sys.dm_exec_query_plan_stats – Provides actual execution plan details for recent queries.
- ☒ Sys.dm_exec_query_plan – Returns estimated plan from cache, not actual execution.
- ☒ Sys.dm_exec_cached_plans – Lists cached plans, but without showing actual runtime stats.

Instructor led labs: Isolate problem areas in poorly performing queries in a SQL Database

Generate actual execution plan
Resolve a suboptimal query plan
Use Query Store to detect and handle regression
Examine Top Resource Consuming Queries report
Force a better execution plan
Use query hints

© Copyright Microsoft Corporation. All rights reserved.

<https://microsoftlearning.github.io/dp-300-database-administrator/Instructions/Labs/10-isolate-problem-areas-poor-performing-queries.html>

References

SQL Server execution plans:

<https://www.red-gate.com/simple-talk/books/sql-server-execution-plans-third-edition-by-grant-fritchey/>

Monitoring performance by using the Query Store:

<https://learn.microsoft.com/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store>

Query processing architecture guide:

<https://learn.microsoft.com/sql/relational-databases/query-processing-architecture-guide>

Index architecture and design:

<https://learn.microsoft.com/sql/relational-databases/sql-server-index-design-guide>

© Copyright Microsoft Corporation. All rights reserved.

Learning Path Recap

In this learning path, we:

We learned how to explore query performance optimization. This involves understanding how plans are generated, understanding query plan warnings and guidance, exploring the Query Store, and learning about the types of query execution plans.

We also learned how to explore performance-based database design. This includes learning the concepts of database normalization, choosing the proper data types for your database, and understanding the types of indexes in SQL Server.

Lastly, learned how to evaluate performance improvements. This involves capturing data from dynamic management views and functions, understanding wait statistics, and understanding index and statistics maintenance.

© Copyright Microsoft Corporation. All rights reserved.



© Copyright Microsoft Corporation. All rights reserved.